Final Report — Email Generation Assistant & Evaluation

Task: build an LLM email assistant (Intent + Key Facts + Tone → professional email), design 3 custom quality metrics, and compare two models on 10 scenarios. **Provider:** Groq (free tier). **Models:** `llama-3.3-70b-versatile` (Meta) vs `openai/gpt-oss-120b` (OpenAI open-weight). **Judge:** `llama-3.3-70b-versatile` @ `temp=0`.

---

1. The Prompt Template (advanced prompting technique)

The assistant uses **three** techniques in one template, sent identically to both models so the comparison isolates the model. (Source: `src/prompts.py`.)

(a) Role-Playing — system prompt

> *You are an expert executive communications assistant who ghost-writes professional emails for busy leaders... You never invent facts, you weave in every fact you are given naturally, and you write tight, skimmable prose with a clear call to action.*

(b) Few-Shot — two compact `input → ideal email` exemplars (a friendly scheduling email and an empathetic apology) are inserted as prior user/assistant turns. They teach the output format and the "include every fact, match the tone" contract by demonstration.

(c) Chain-of-Thought (hidden) — the task turn appends:

> *Before writing, plan silently (do NOT show this): 1. recipient & goal; 2. check off each key fact and where it fits; 3. calibrate vocabulary/warmth/urgency to the tone; 4. structure (subject, greeting, tight body, clear CTA, sign-off). Then output ONLY the final email.*

Why this combination: role-play sets register; few-shot locks format and the fact/tone contract; hidden CoT raises fact coverage and structure while keeping the output clean for both a reasoning model (gpt-oss) and a non-reasoning one (llama).

---

2. The Three Custom Metrics (definitions & logic)

Design principle: **match the measurement technique to the nature of the quality** — measure what is objective, judge what is subjective. All three return 0–100. (Source: `src/metrics.py`.)

#	Metric	Focus	Technique	Logic	Why it matters

1	**Fact Recall**	Fact Recall / Specificity	Structured LLM check (per-fact yes/no) → scored in Python	`present / total × 100`	Dropping a required fact (price, date, order #) is a functional failure regardless of polish. Most important correctness metric.
2	**Tone Accuracy**	Tone Accuracy	LLM-as-judge, 1–5 rubric	`rubric × 20`	Tone is subjective/holistic; a keyword heuristic can't tell "warm" from "curt".
3	**Conciseness & Fluency**	Conciseness + Grammar/Fluency	**Hybrid**: pure-Python length analysis vs the human reference **+** LLM fluency rubric	`0.5 × length_band(len/ref_len) + 0.5 × (fluency 1–5 × 20)`	A correct, on-tone email still fails if it's bloated or clumsy.

For Metric 3, the **Python** half scores how close the email's word count is to the *ideal human reference* length (full marks within 0.80–1.25× the reference, tapering outside), with a sentence-length sanity check; the **LLM** half scores grammar/natural phrasing only.

Judge integrity: one fixed neutral judge scores *both* candidates at `temperature=0` (removes self-preference bias, deterministic). The model-dependent parts share **one** structured judge call per email for token efficiency.

---

3. Raw Evaluation Data

Per-scenario scores (full data: `results/scores.csv`, `results/scores.json`; emails: `results/generations.json`).

llama-3.3-70b

Scenario	Tone	Fact	Tone	Concise/Flu	Overall
S01 sales follow-up	professional/friendly	100	100	83.5	94.5
S02 RFP request	formal	100	100	75.6	91.9
S03 outage apology	empathetic	100	100	73.6	91.2
S04 reconnect colleague	casual/warm	100	100	100	100

S05 deadline escalation	urgent	100	100	100	100
S06 partner pitch	persuasive	100	100	81.3	93.8
S07 invoice reminder	polite/concise	100	100	100	100
S08 interview thank-you	appreciative	100	100	88.7	96.2
S09 policy announcement	clear/neutral	100	100	100	100
S10 decline meeting	courteous	100	100	86.0	95.3

gpt-oss-120b

Scenario	Tone	Fact	Tone	Concise/Flu	Overall
S01 sales follow-up	professional/friendly	100	100	100	100
S02 RFP request	formal	100	100	100	100
S03 outage apology	empathetic	100	100	100	100
S04 reconnect colleague	casual/warm	**75**	100	100	91.7
S05 deadline escalation	urgent	100	100	100	100
S06 partner pitch	persuasive	100	100	83.8	94.6
S07 invoice reminder	polite/concise	100	100	100	100
S08 interview thank-you	appreciative	100	100	95.5	98.5
S09 policy announcement	clear/neutral	100	100	92.7	97.6
S10 decline meeting	courteous	100	100	100	100

Aggregate (mean over 10 scenarios; ± = population std)

Model	Fact Recall	Tone Accuracy	Concise/Fluency	Overall

llama-3.3-70b	**100.0** (±0.0)	100.0 (±0.0)	88.9 (±9.8)	96.3 (±3.3)
gpt-oss-120b	97.5 (±7.5)	100.0 (±0.0)	**97.2** (±5.4)	**98.2** (±2.8)

---

4. Comparative Analysis (Section 3)

Which model performed better? gpt-oss-120b, by **+1.9** overall (98.2 vs 96.3). It split the metrics with Llama: Llama won **Fact Recall** (100.0 vs 97.5) and tied **Tone Accuracy** (both perfect, 100.0), while gpt-oss won **Conciseness & Fluency** decisively (**97.2 vs 88.9, +8.3**). Both models nailed tone on every scenario, so the comparison came down to facts vs concision.

Biggest failure mode of the lower-performing model (llama-3.3-70b): verbosity. The data is unambiguous — Llama's fluency was perfect (5/5 grammar everywhere), but it consistently **over-wrote**. Its generated emails averaged **1.42× the reference length** (gpt-oss: 1.13×), and the worst offenders were exactly the scenarios where it lost points: - **S03 outage apology** — 177 words vs a 97-word reference (**1.83×**), length score 47.3. - **S02 RFP request** — 152 words vs 86 (**1.77×**), length score 51.3. - **S06 partner pitch** — 159 words vs 99 (**1.61×**), length score 62.6.

So Llama writes *well* but not *tightly*: it pads with framing and restatement, which is a real cost for busy email readers. gpt-oss's only blemish was the mirror image — on **S04** it dropped one required fact ("recently joined Stripe"), giving 75 on that scenario; it occasionally trades a detail for brevity.

Production recommendation: `gpt-oss-120b`, with a fact-completeness guardrail. It wins on quality (98.2), is materially **cheaper** ($0.15/$0.60 vs $0.59/$0.79 per 1M tokens in/out), **faster** (≈500 vs ≈280 tokens/sec on Groq), and produces send-ready, concise emails. Its one risk is the most business-critical dimension — fact omission — so I'd pair it with the cheap programmatic **Fact Recall check we already built** (or a higher reasoning effort) to auto-flag the rare miss before send. That combination gives gpt-oss's brevity and cost with Llama's reliability.

Choose `llama-3.3-70b` instead only if fact completeness must be guaranteed with *zero* tooling around the model and mild verbosity is acceptable — e.g., long-form drafts a human edits anyway.

---

5. Caveats & critical notes

- **Small sample (n=10).** The +1.9 overall gap is within the run-to-run noise you'd expect from an LLM judge, so it is *directional, not conclusive*; the **conciseness gap (+8.3) is the robust, repeatable finding**. Tone saturated at 100 for both — a harder tone rubric (or adversarial scenarios) would discriminate better next iteration.
- **LLM-judge limits.** Even a neutral judge at `temp=0` carries verbosity/leniency biases; Fact Recall is the most trustworthy metric here because it's a verifiable yes/no, not a holistic score.
- **Reproducibility.** All sampling params, model IDs, and timestamps are logged into `results/summary.json` and `results/scores.json`.
- **Next steps:** multi-seed runs for confidence intervals, a second independent judge for agreement (Cohen's κ), and harder scenarios (longer fact lists, conflicting constraints) to stress Fact Recall.